

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI Todo Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 3
 - 2.1 CLI Todo Manager Traceability Matrix 6
 - 2.2 CLI Command Interaction Sequence 7
 - 2.3 CLI Validation Workflow 8
 - 2.4 Todo Data Model 9
 - 3. Detailed Technical Specifications & Business Rules 11
 - 3.1 Requirements Traceability 11
 - 3.2 Security Rules 12
 - 3.3 Data Models 12
 - 4. Project Governance & Structural Gaps 12
 - 4.1 Structural Gaps 12
 - 4.2 Remediation & Workflow 13
 - 5. Technical & Domain Glossary (Terminology Reference) 13

CLI Todo Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI Todo Manager is a Python-based command-line utility designed for task lifecycle management. It utilizes a local JSON file (`~/ . todos . json`) as its primary data store to persist task IDs, descriptions, and completion statuses, providing both human-readable and JSON output formats.

1.2 Maturity Assessment

The project is READY for execution. While there are minor structural gaps regarding formal high-level acceptance criteria and a tracking checklist, the functional requirements and test scenarios are explicitly defined and mapped, ensuring a clear path to operational validation.

1.3 Technical Stack

- Python 3.8+

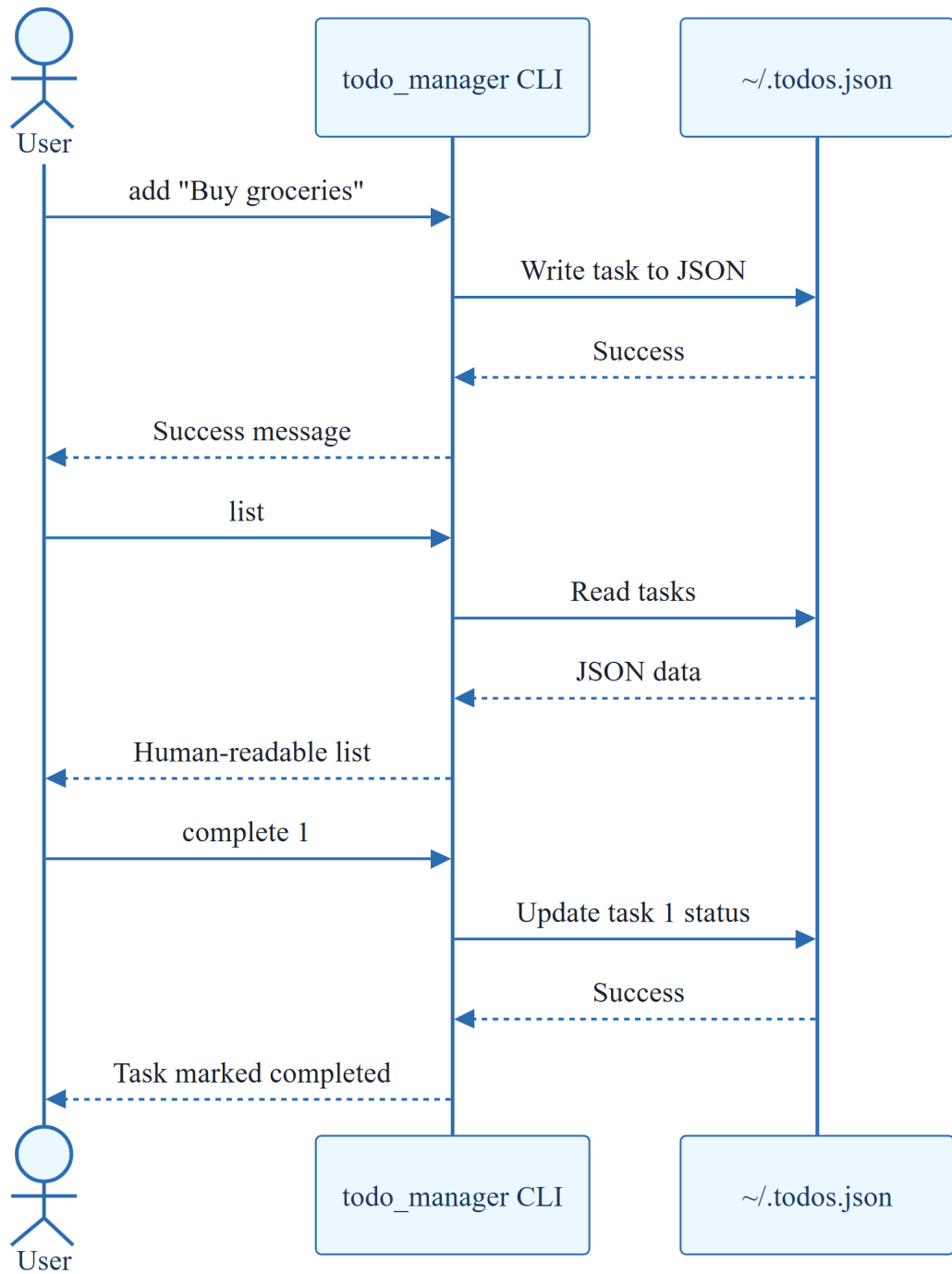
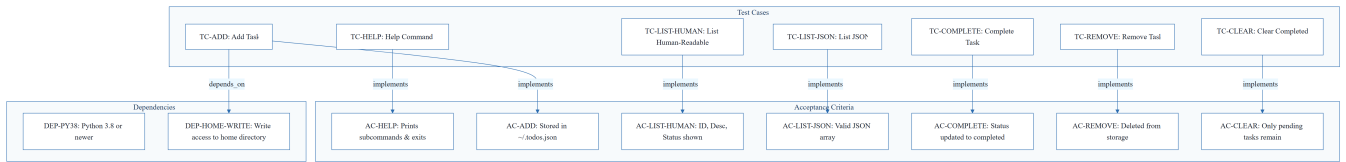
1.4 Architectural Constraints

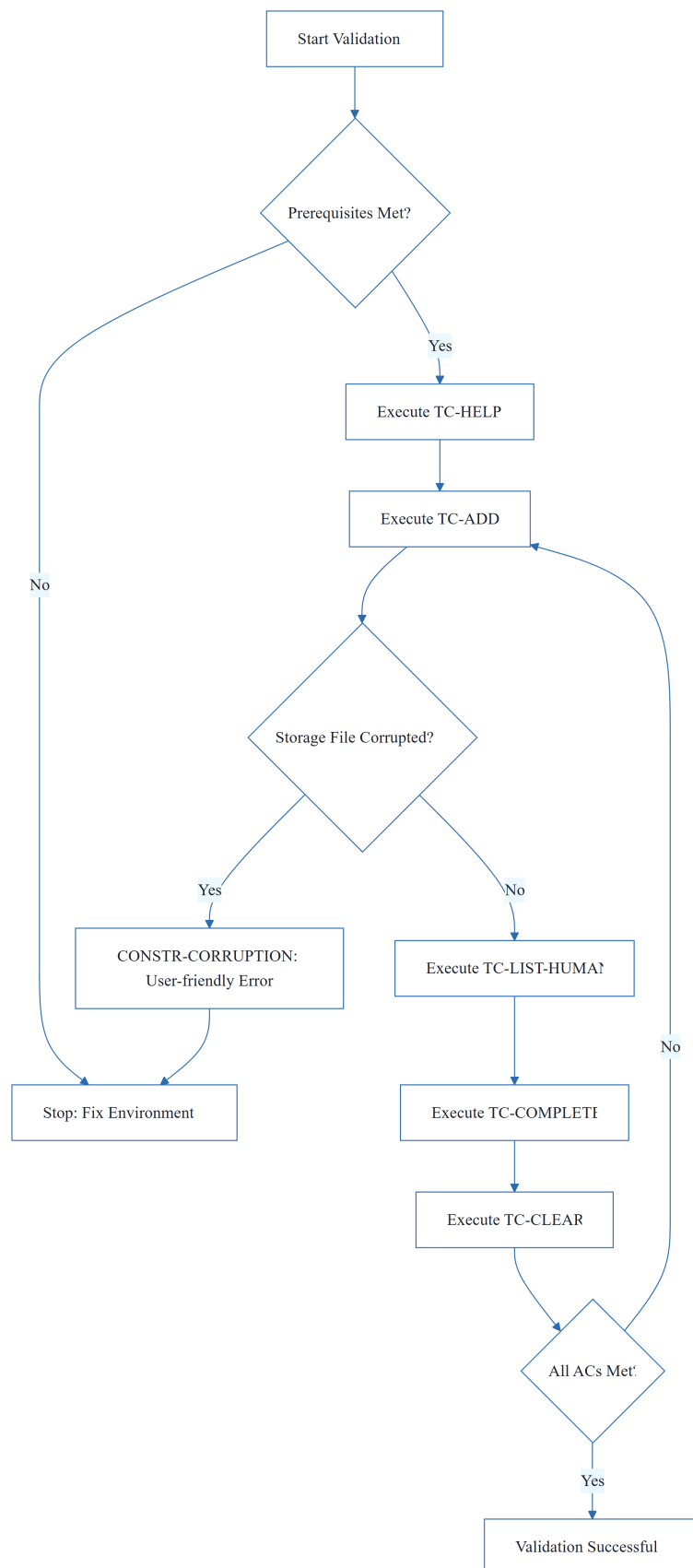
- **Storage:** Local persistence via `~/ . todos . json`.
- **Data Integrity:** CLI must terminate with a user-friendly error upon detecting storage corruption to prevent accidental overwriting.
- **Output Modes:** Support for both human-readable and valid JSON array formats.
- **File System:** Mandatory write access to the user's home directory.

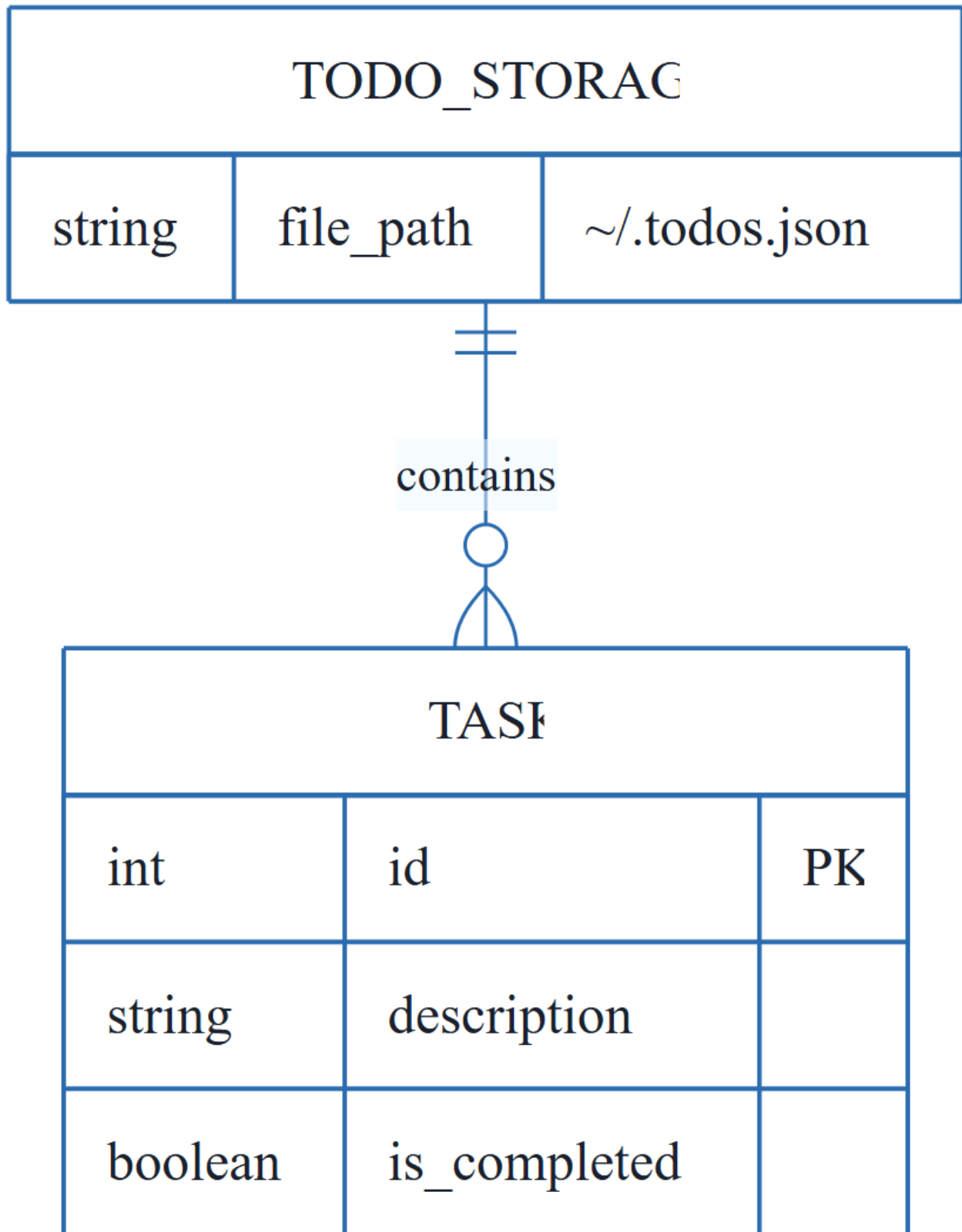
1.5 Critical Dependencies

- Python 3.8 runtime.
- Home directory write permissions for `~/ . todos . json`.
- Referential integrity between task IDs and storage entries for 'complete' and 'remove' operations.
- External specification alignment with `data-model.md` and `contracts/commands.md`.

2. Architecture Workflows



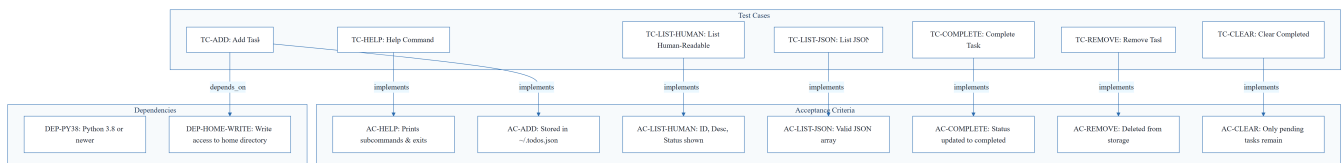




& Visual Diagrams

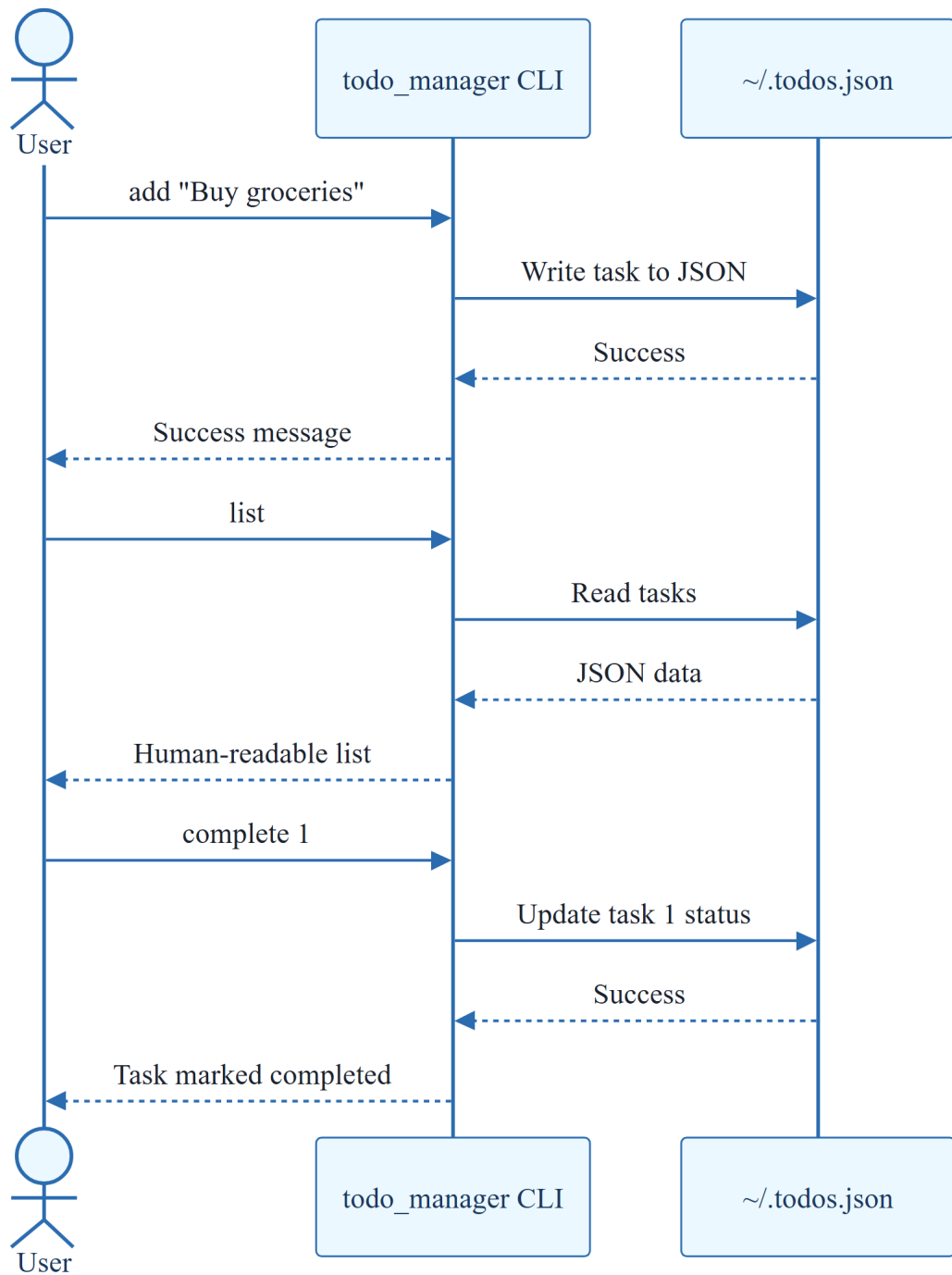
2.1 CLI Todo Manager Traceability Matrix

Maps test cases to their respective acceptance criteria and system dependencies to ensure full validation coverage.



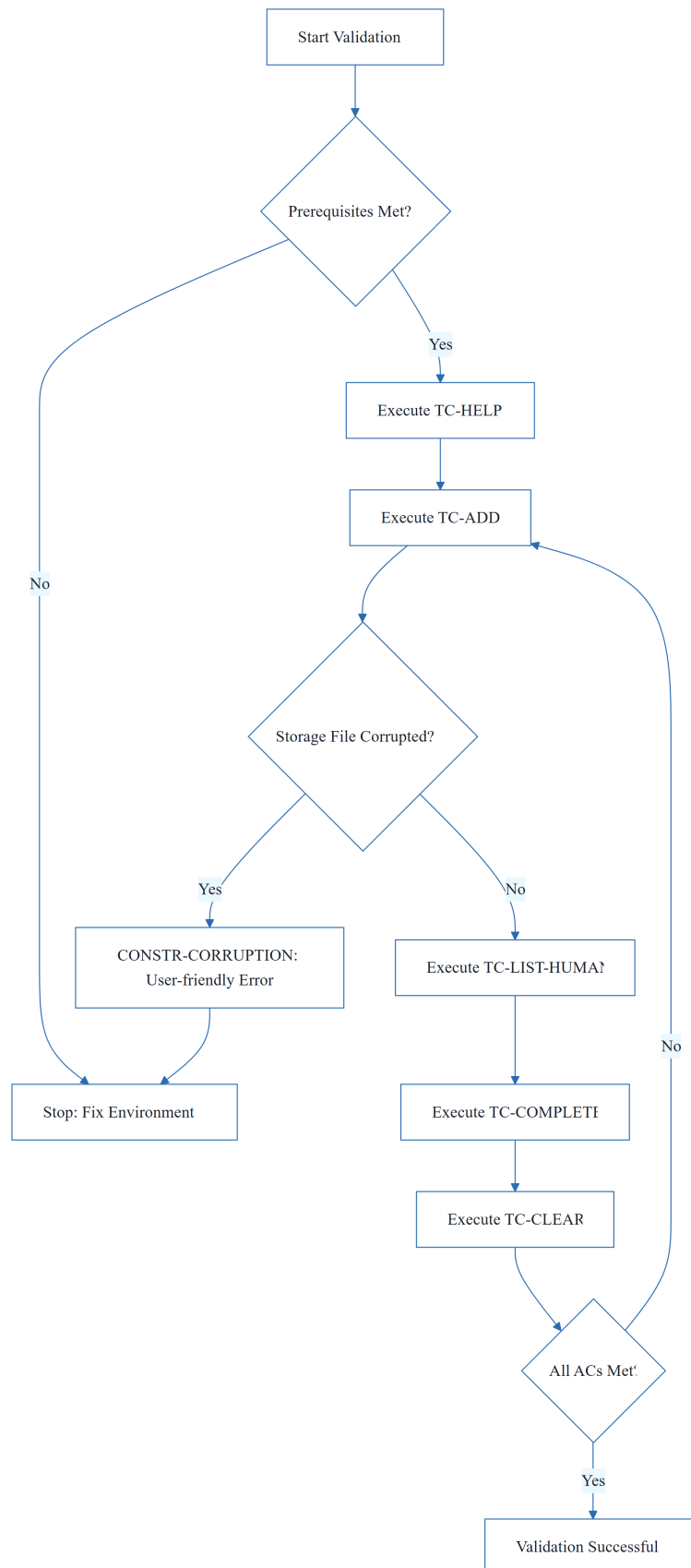
2.2 CLI Command Interaction Sequence

Models the interaction between the User, the CLI Application, and the JSON storage file for core operations.



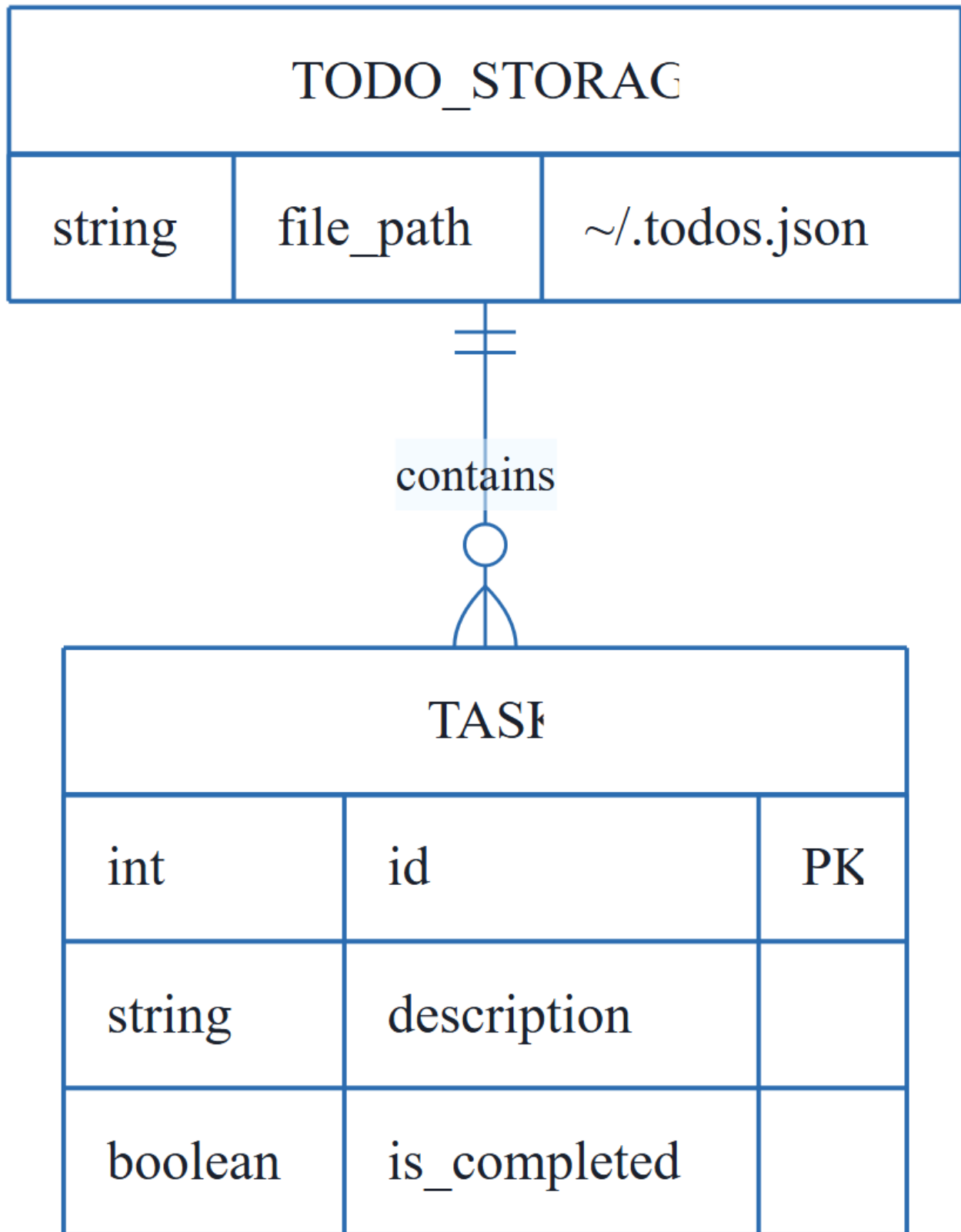
2.3 CLI Validation Workflow

The operational logic for validating the CLI, including error handling for file corruption.



2.4 Todo Data Model

Represents the structure of the tasks stored in the JSON file as implied by the validation guide.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

Identifier	Type	Description	Acceptance Criterion / Dependency
DEP-PY38	Dependency	Python 3.8 or newer	N/A
DEP-HOME-WRITE	Dependency	Write access to home directory	N/A
TC-HELP	Test Case	Confirm the command help is available: <code>python -m todo_manager --help</code>	AC-HELP
AC-HELP	Acceptance	CLI prints available subcommands and exits successfully	N/A
TC-ADD	Test Case	Add a task: <code>python -m todo_manager add "Buy groceries"</code>	AC-ADD
AC-ADD	Acceptance	Task is stored in <code>~/ .todos . json</code> and success message is shown	N/A
TC-LIST-HUMAN	Test Case	List tasks in human-readable mode: <code>python -m todo_manager list</code>	AC-LIST-HUMAN
AC-LIST-HUMAN	Acceptance	Task appears with ID, description, and pending/completed status	N/A
TC-LIST-JSON	Test Case	List tasks as JSON: <code>python -m todo_manager list --json</code>	AC-LIST-JSON
AC-LIST-JSON	Acceptance	CLI prints valid JSON containing the tasks array	N/A
TC-COMPLETE	Test Case	Complete a task: <code>python -m todo_manager complete 1</code>	AC-COMPLETE
AC-COMPLETE	Acceptance	Task is marked completed and later listings show updated status	N/A
TC-REMOVE	Test Case	Remove a task: <code>python -m todo_manager remove 1</code>	AC-REMOVE
AC-REMOVE	Acceptance	Task is deleted from storage	N/A

Identifier	Type	Description	Acceptance Criterion / Dependency
TC-CLEAR	Test Case	Clear completed tasks: <code>python -m todo_manager clear</code>	AC-CLEAR
AC-CLEAR	Acceptance	All completed tasks are removed and pending tasks remain	N/A
CONSTR-CORRUPTION	Constraint	If storage file is corrupted, CLI should stop with user-friendly error instead of overwriting	N/A

3.2 Security Rules

- **File System Access:** The application requires explicit write permissions to the user's home directory to maintain the `~/ .todos.json` file.
- **Data Integrity:** To prevent data loss, the system must implement a "fail-fast" mechanism upon detecting JSON corruption (CONSTR-CORRUPTION).

3.3 Data Models

- **Storage Format:** JSON.
- **Storage Path:** `~/ .todos.json`.
- **Task Entity:**
 - `id` (Integer): Unique identifier.
 - `description` (String): Task text.
 - `is_completed` (Boolean): Completion status.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	LOW	The document contains 'Expected outcomes' within the test cases, but no high-level business acceptance criteria section.
Checkboxes Checklist	MEDIUM	Convert the 'Validate the CLI' steps into a formal checklist for execution tracking.

Gap	Priority	Remediation Advice
Security & Performance Constraints	LOW	Define performance expectations for large todo lists or security constraints for the JSON file.
Open Questions & Uncertainties	LOW	None needed unless specific edge cases are identified during validation.

4.2 Remediation & Workflow

The identified gaps are primarily documentation-related and do not block the current execution phase. Remediation should focus on converting the existing test cases into a formal validation checklist to improve tracking during the QA phase.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
ID	BUSINESS_DOMAIN	AC-LIST-HUMAN	A unique numeric reference used to target a specific entry for completion or removal operations.
JSON	TECHNICAL_STACK	AC-LIST-JSON	The structured data format used for both persistent storage in the home directory and machine-readable output.
Python 3.8	TECHNICAL_STACK	DEP-PY38	The minimum required runtime environment version for executing the command line interface.